

Development of a Deep Learning Model for
Synthetic vs. Real Image Classification

Bernardo Gandara - Ignacio Alarcon Varela

A Report Submitted in Partial Fulfilment
Of the requirements for the
Degree of
BSc in Computing and IT (4th Year)

May 2025



Supervisor: Dr Muhammad Iqbal

Abstract

The advancement of generative AI technologies has made it increasingly difficult to distinguish synthetic images from authentic ones. This capstone project addresses the challenge by developing a binary image classification model using deep learning techniques to differentiate AI-generated images from real photographs. Guided by the CRISP-DM methodology, we employed the DeepGuardDB dataset, consisting of 13,000 balanced image samples, evenly split between real and synthetic sources. We implemented and compared three Convolutional Neural Network (CNN) architectures through transfer learning, standardising input pipelines and integrating custom classification heads. Following a performance evaluation across multiple metrics, the best-performing model was selected for further optimisation using established techniques such as hyperparameter tuning and fine-tuning. This project demonstrates how deep learning can contribute to maintaining information integrity in media, journalism, and digital content platforms.

Table of Content

1. INTRODUCTION	5
1.1. Project Concept.....	5
1.2. Roles & Responsibilities.....	5
2. METHODOLOGY	7
2.1. CRISP-DM Model	7
3. BUSINESS UNDERSTANDING (CRISP-DM Stage 1)	10
3.1. Problem Definition.....	10
3.2. Business Objective	10
3.3. Business Success Criteria	10
3.4. Project Resources.....	11
4. DATA UNDERSTANDING (CRISP-DM Stage 2).....	12
4.1. Ethical Considerations	13
4.2. Exploratory Data Analysis (EDA)	14
5. DATA PREPARATION (CRISP-DM Stage 3)	19
5.1. Construction of Training, Validation and Testing Sets	19
5.2. Data Augmentation	20
5.3. Data Preprocessing for each Model	21
6. MODELLING (CRISP-DM Stage 4)	22
6.1. Convolutional Neural Networks	22
6.1.1. CNN 101 Architecture	22
6.2. Transfer Learning.....	23
6.3. VGG-16 Architecture.....	24
6.3.1. Degradation Problem	24
6.4. ResNet-50 Architecture	25

6.4.1. Residual Block and Skip Connection	25
6.5. EfficientNet Architecture	26
6.5.1. Compound Scaling.....	27
6.5.2. Main Components.....	27
6.6. Custom Classification Head.....	30
6.7. Training Optimisation Strategies	31
6.8. Conclusion	32
7. EVALUATION (CRISP-DM Stage 5)	33
7.1. Confusion Matrices	34
7.2. Classification Report.....	35
7.3. Training Dynamics.....	36
7.4. Model Selection	37
7.5. Model Optimisation: Hyperparameter Tuning and Fine-Tuning.....	37
7.5.1. Hyperparameter Optimisation (HPO) - Keras Tuner.....	37
7.5.2. Fine-Tuning.....	38
7.6. Final Evaluation	39
8. Deployment (CRISP-DM Stage 6)	41
9. Conclusion	43
Appendix.....	44
Word Count.....	44
GitHub Repository Link	44
Original Dataset Link.....	44
Reflective Journal	44
References.....	47

1. INTRODUCTION

1.1. Project Concept

The concept of this project is to develop and evaluate a Convolutional Neural Network (CNN)-based model capable of distinguishing AI-generated images from authentic photographs. CNNs are probably the most popular of all neural network architectures. This is mainly because, although they work in many domains, they are particularly good at dealing with images. A CNN can capture the spatial dependencies of images since it processes them as matrices and analyses entire chunks of an image at a time, depending on the size of the filter. (Saleh, 2020)

By carefully preparing and preprocessing a balanced dataset, implementing multiple pre-trained architectures through transfer learning, and optimising the best-performing model via hyperparameter tuning and fine-tuning strategies, this project aims to build a robust binary image classifier. Our objective is to contribute to the development of reliable detection tools by enhancing classification accuracy and generalisation in the context of synthetic media recognition.

1.2. Roles & Responsibilities

The CRISP-DM methodology provided a structured framework that facilitated the division of tasks across the different stages of the project. Despite this division, both team members maintained constant collaboration and shared awareness throughout all phases, ensuring consistent alignment and effective execution.

Bernardo Gandara was primarily responsible for Stage 1 (Business Understanding), Stage 2 (Data Understanding), the theoretical foundation of Stage 4 (Modelling), and the optimisation process in Stage 5 (Evaluation). His work involved researching current industry needs to define project objectives, exploring and analysing the dataset through exploratory data analysis (EDA), investigating the anatomy and function of various CNN architectures, and implementing hyperparameter tuning and fine-tuning strategies to optimise the selected model.

Ignacio Alarcon Varela led Stage 3 (Data Preparation), the practical implementation of Stage 4 (Modelling), Stage 5 (Evaluation), and Stage 6 (Deployment). His contributions included applying

data preprocessing and augmentation techniques, ensuring compatibility with each architecture, executing model training and interactive testing, performing evaluation using classification reports and confusion matrices, and deploying the final model configuration.

Although responsibilities were distributed, all strategic decisions and implementations were carried out with full involvement from both members. Regular meetings, collaborative coding sessions, and open communication were essential to navigate technical challenges and achieve the project's objectives efficiently.

Business Understanding

The goal in this stage is to define and understand the industry needs and objectives, since those objectives will become the objectives of the project. Within this phase, we gathered information to define a project plan and clearly state data mining goal and how we plan to reach it.

Data Understanding

Now the goals and success criteria of our activity are clear, we gathered relevant data for our project. The first task of this phase was to start acquiring from the dataset mentioned in the resources part of the previous stage.

Data Preparation

The modelling technique we are going to apply to our data will have specific requirements. The objective at this stage is to prepare our data in a way that is suitable for our chosen architecture. We may need some basic transformation activities on our raw data to be performed.

Modelling

We try to get knowledge from our data, answering the questions defined within the business understanding phase. What is relevant to understand on this phase is that it should not be considered a linear and sequential phase, but rather a cyclical one. The way it usually performs is through the following three steps: *defining a data modelling strategy, model assumption evaluation, model assessment*.

Evaluation

The evaluation phase is the one where we look for a validation of the results coming from our modelling activities. This phase can be split into two main questions:

- Is the model performing adequately?
- Is the model answering the question originally posed?

These questions involve the identification of a proper set of metrics to establish if the model developed possesses the desired properties.

Deployment

We are now reaching the final phase, and we are going to implement our models into the production system. However, if the previous phase does not go well, it would be pointless to develop a deployment plan. This is when we understand that the CRISP-DM methodology is an iterative process.

3. BUSINESS UNDERSTANDING (CRISP-DM Stage 1)

3.1. Problem Definition

The exponential growth of generative AI, especially in the form of *deepfakes*, has made it increasingly more challenging to distinguish real images from fake ones. Deepfakes refer to hyper-realistic content created using deep learning models (especially Generative Adversarial Networks - GANs) which can fabricate or manipulate media to appear convincingly authentic to the human eye. (Babcock and Bali, 2021).

While these technologies provide many productive and creative applications, they also pose a particular high risk when used to spread misinformation.

An increasing concern is the use of AI-generated images in the context of fake news and misleading narratives. How simple and effortless it is to create new, lifelike identities is demonstrated by platforms such as *thispersondoesnotexist.com*. Such synthetic content can be incorporated into propaganda or fake news articles, influencing public opinion and media credibility. As the line between authentic and generated images becomes blurred, the need for detection tools becomes crucial to safeguard information integrity.

3.2. Business Objective

Our project aims to develop a robust deep learning model capable of accurately classifying whether an image is AI-generated or real. By doing so, we contribute to the growing demand for verification tools within journalism, social media platforms and content moderation systems. The goal is to support efforts in identifying synthetic content at scale and preventing the viral spread of deceptive media.

3.3. Business Success Criteria

The project will be considered successful if the final model demonstrates strong and reliable performance in distinguishing AI-generated images from real ones. Success will be evaluated based on the following criteria:

- The model continuously scores highly across classification standard metrics such as accuracy, precision, and recall.

- It effectively minimises both false positives (fake images misclassified as real) and false negatives (real images misclassified as fake).
- It generalises well across different sources and types of image content, not just the training data.
- It shows potential for integration into real-world applications such as digital content verification systems, news validation tools, or social media moderation platforms.

3.4. Project Resources

To ensure an efficient implementation of our image classification project, we identified the following resources:

- **Hardware**

While development was carried out on personal devices (MacBook Pro M2 & MacBook Pro M1 max), all practical implementation tasks were executed on Google Colab, leveraging its cloud-based infrastructure. Specifically, we used a A100 GPU, which offers higher performance for deep learning tasks compared to standard CPUs and T4 GPUs.

- **Software**

We used the Python programming language along with the relevant libraries necessary to support all stages of the project, including exploratory data analysis (EDA), data preparation, modelling, hyperparameter tuning, model evaluation and deployment.

All development was carried out in a Google Colab notebook, which provided an environment with seamless access to GPU processing. Deployment was developed on Visual Studio Code using *streamlit* as our chosen framework to create a web application.

- **Dataset**

We selected the DeepGuardDB dataset, which contains 13,000 images evenly split between real and AI-generated (fake) samples. A detailed description of this dataset and our analysis is provided in the Data Understanding section.

4. DATA UNDERSTANDING (CRISP-DM Stage 2)

The DeepGuardDB dataset, a carefully created collection designed to aid studies in distinguishing between real and artificial intelligence-generated photos, was used for our experiment. The 13,000 photos in this collection are split equally between 6,500 synthetic images created by text-to-image (T2I) models and 6,500 real photographs.

According to the dataset abstract:

- Real Images: These were sourced from two well-established datasets:
 - MS-COCO (Microsoft Common Objects in Context)
 - Flickr30k

Both datasets are renowned for their diverse and rich visual content, providing a robust foundation for training and evaluation.

- Synthetic Images: The AI-generated images were produced using three state-of-the-art T2I generation platforms:
 - Stable Diffusion 3
 - Imagen
 - DALL-E 3

To ensure a fair comparison, a textual description of the real image was used as a prompt to create the corresponding synthetic image. This method allowed both the real and generated versions to represent the same subject or scene, making it more challenging for a model to differentiate between them based solely on visual content.

The dataset is organised into separate folders for each T2I platform, with subfolders for real and synthetic images. Additionally, a comprehensive JSON file accompanies the dataset, mapping each pair of images (real and synthetic) to its corresponding prompt and associated metadata, such as the generation platform. This structure facilitates traceability and aids in the analysis of the generation process.

To simplify the organization and improve accessibility, we restructured the original dataset. While the initial *DeepGuard_v1* directory grouped images by generation platform (DALL-E, Image,

Stable Diffusion and Glide), each with separate “real” and “fake” subfolders, we created a new *DeepGuard_v2* folder where all real images were grouped into a single “real” folder and all AI-generated images into a single “fake” folder. This unified structure provided a cleaner and more efficient foundation for the subsequent stages of the project.

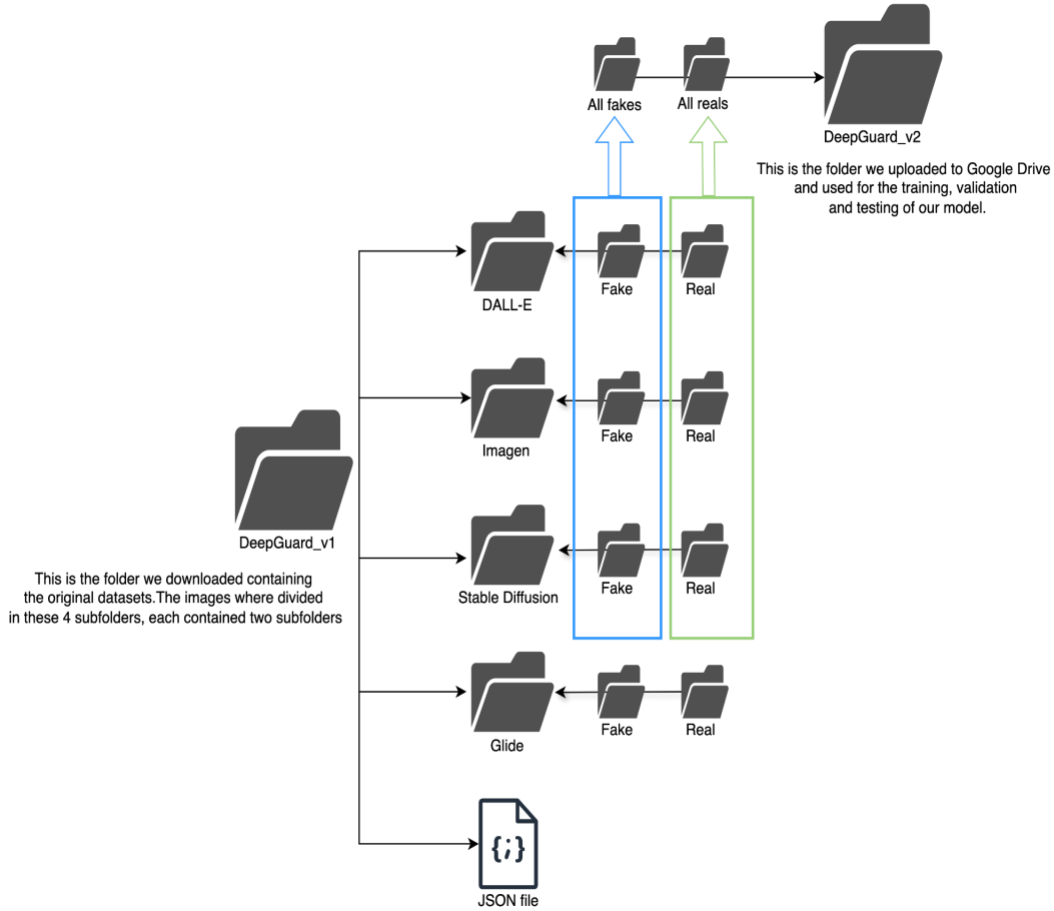


Figure 4-1 - Distribution of files in the original dataset (*DeepGuard_v1*) and restructured dataset (*DeepGuard_v2*)

4.1. Ethical Considerations

The DeepGuardDB dataset, used in this study to differentiate real from AI-generated images, is ethically and legally suitable for academic research. Its real image subset is sourced from MS-COCO, a widely adopted dataset distributed under the Creative Commons Attribution 4.0 License (CC BY 4.0). This license permits sharing, adaptation, and use—including for research purposes—provided appropriate credit is given. According to the MS-COCO Terms of Use, the dataset may be freely used for non-commercial research and development. Therefore, our usage complies fully with licensing terms and raises no ethical concerns. (cocodataset.org, n.d.) (Creative Commons, 2018)

4.2. Exploratory Data Analysis (EDA)

To better understand the dataset and ensure its suitability for training, the following key questions guided the EDA process:

1. Are the image counts consistent across all folders?

Identifying any class imbalance is crucial. If one AI model folder contains more images than others, the model may become *biased* toward that specific generator and fail to generalise well across others. Balanced data helps ensure the model learns differentiating features across all image sources equally.

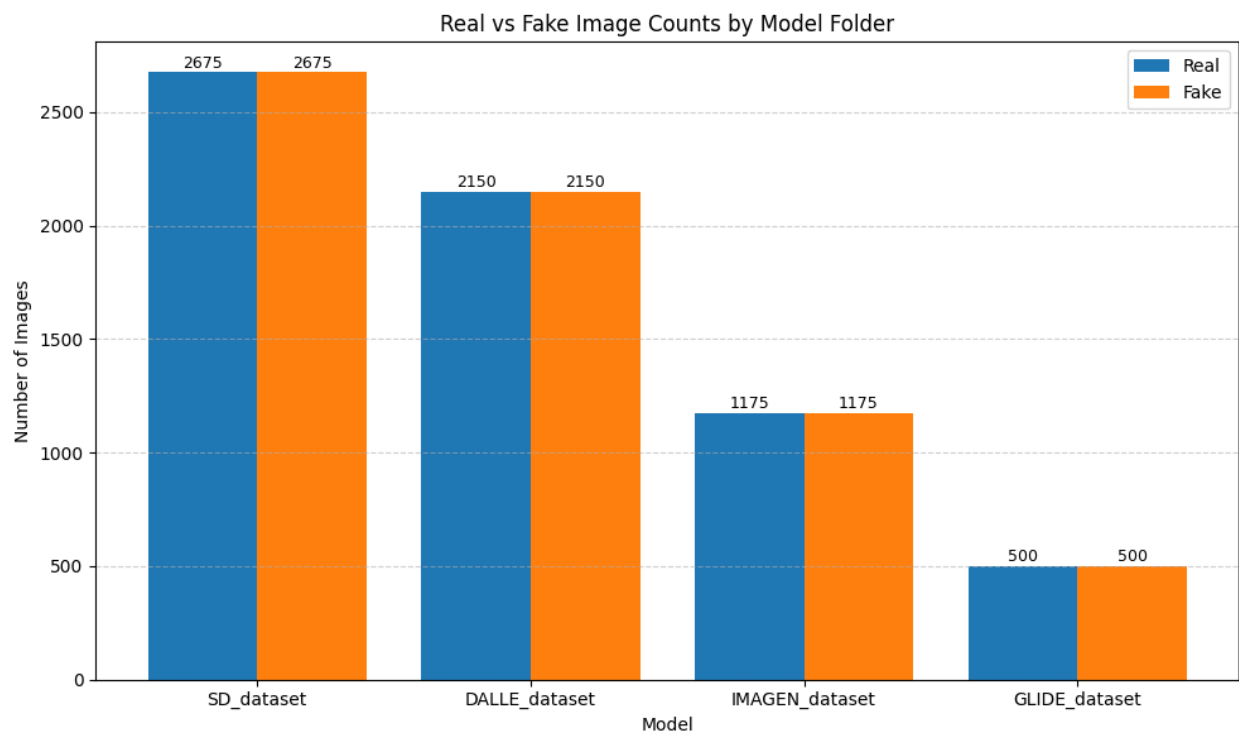


Figure 4-2 - Real vs Fake Image Counts by Model Folder

As we can see in the *Figure 4.2*, there is a balanced number of images within each individual folder (equal real and fake samples), which is ideal. However, we can notice a clear imbalance between folders *SD_dataset* (containing 2675 images per class) and *GLIDE_dataset* (containing 500 images per class). This disparity can cause the model to learn more from Stable Diffusion data and perform better at identifying fakes from that specific AI-generator compared to others with fewer training samples. Therefore, we anticipate that the model may generalise unevenly across different

AI-generators, especially underperforming on those smaller datasets like GLIDE. It is necessary to address this imbalance to ensure fair performance across all sources.

2. Are the image dimensions consistent across the dataset?

Deep learning models need all input images to have the same dimensions. The variations in dimensions create training inconsistencies which need to be resolved during the preprocessing stage. (Antoniadis, 2022)

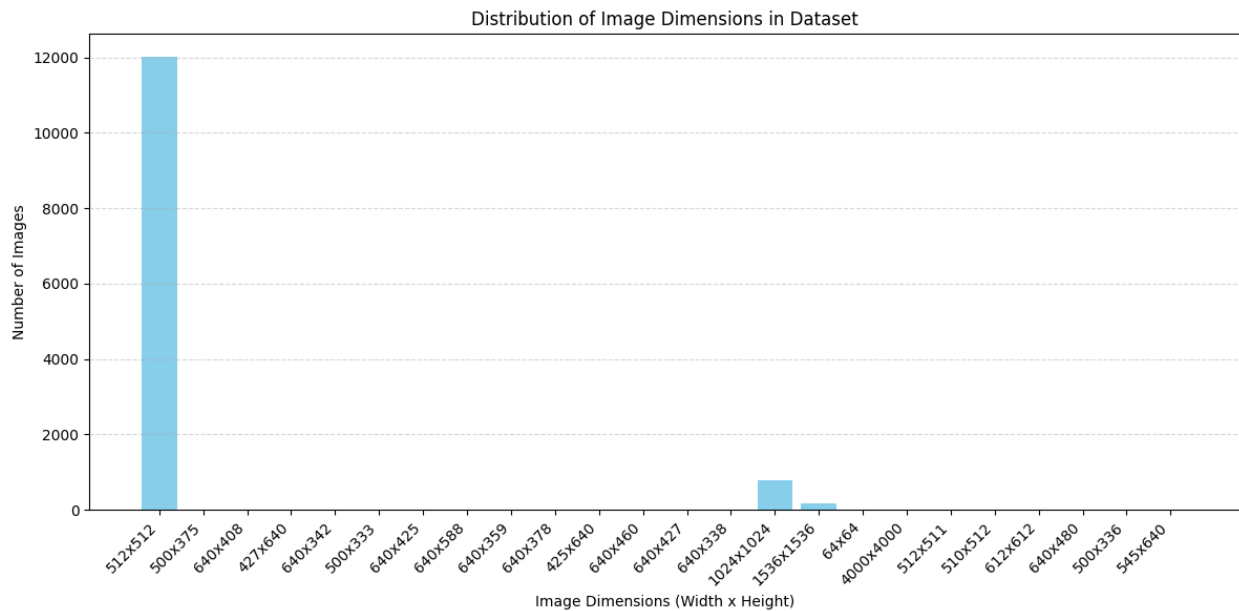


Figure 4-3 - Distributions of image dimensions in the dataset.

Figure 4.3 indicates that most images (12,020) are sized 512x512, establishing it as the leading format in the dataset. A limited number (779 images) are sized 1024x1024, and even less adhere to other non-standard dimensions, many of which are seen only once.

While this inconsistency is slight, it is crucial to standardise all images to a consistent size during preprocessing. Neglecting to address this may negatively affect model performance because of inconsistent input dimensions, particularly since deep learning frameworks need uniform input sizes. Given the small number of outliers, standardising all images could be a practical solution that minimises loss.

3. Do the file types and format match across all files?

Ensuring consistency in image formats (e.g., JPEG, PNG) can avoid compatibility issues during preprocessing.

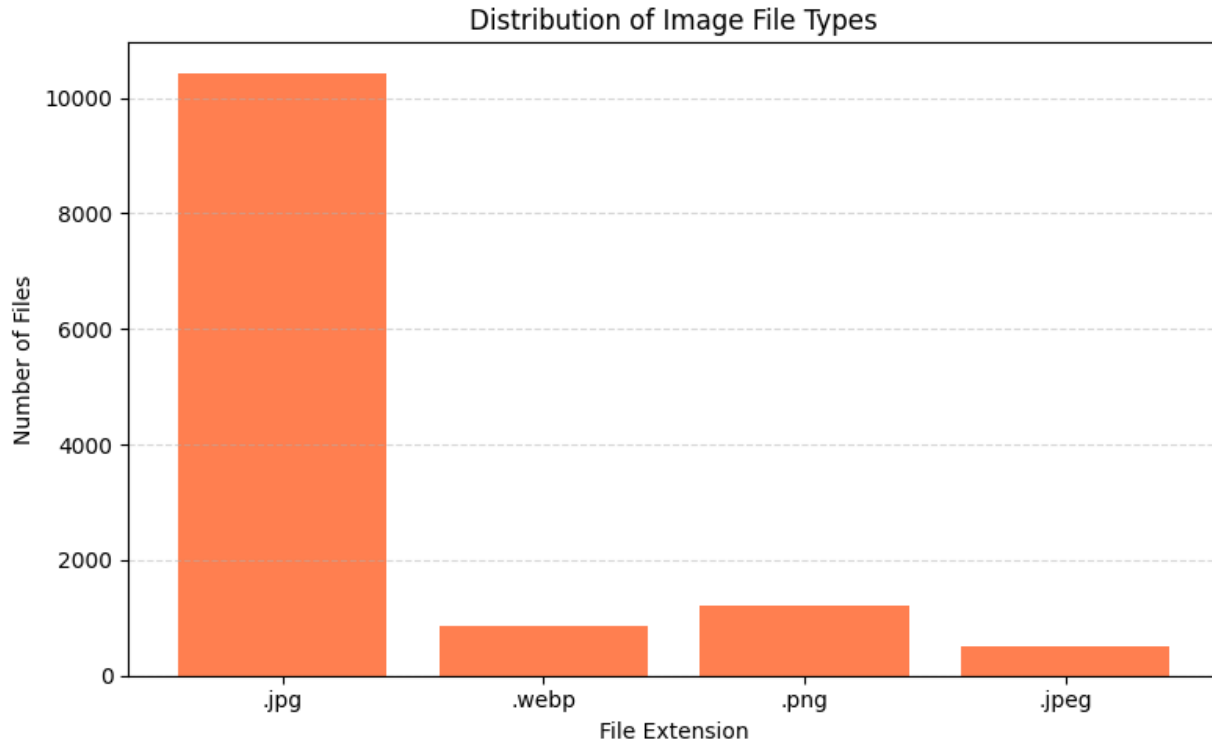


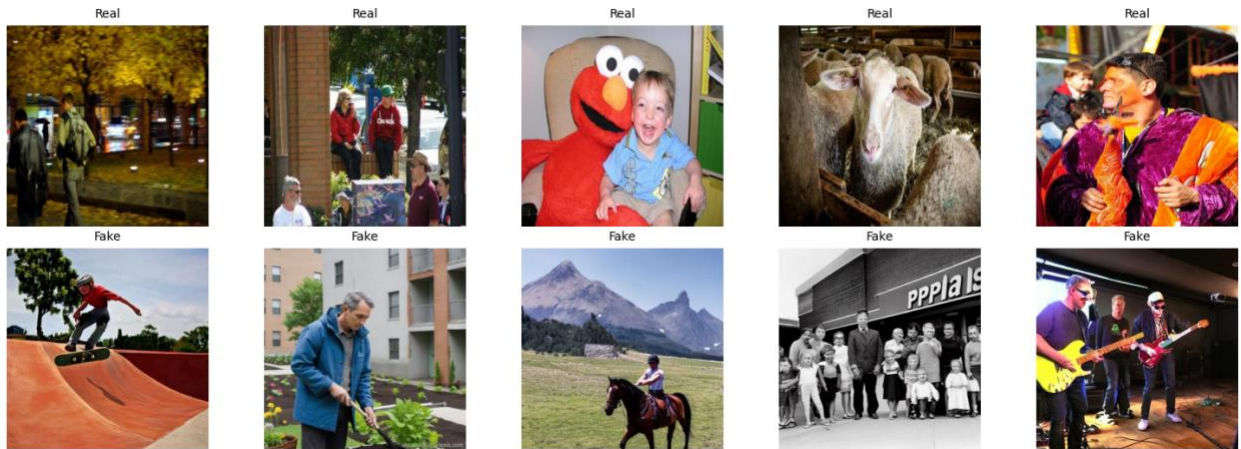
Figure 4-4 - Distributions of Image File Types

Figure 4.4 indicates that the dataset is primarily composed of .jpg images, with over 10,000 files in this format. However, several other formats are also present, including .png, .jpeg, and webp. To ensure consistency and avoid unexpected behaviour, it is recommended to convert all images to a common format during the data preparation stage. Using a uniform image format like .jpg can simplify the workflow, especially when utilizing libraries that expect consistent input formats.

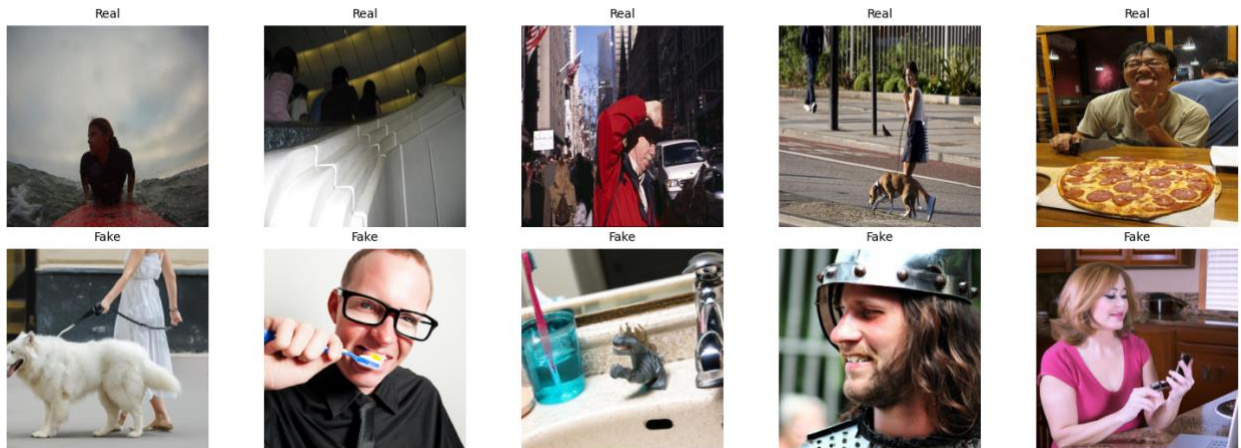
4. Getting a quick look of the samples

To gain an initial visual understanding of the dataset, we selected a few sample images from both the real and fake categories in each folder. This quick inspection helps verify that the images are correctly organized, visually coherent, and aligned with their labels. It also offers insight into the differences and similarities between real and AI-generated images across different models.

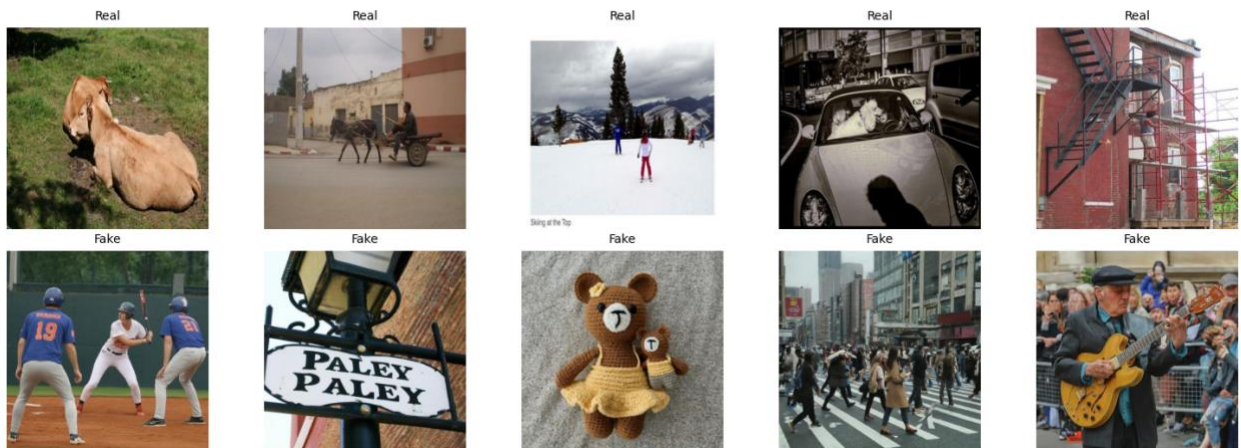
Sample Images from SD_dataset Folder



Sample Images from DALLE_dataset Folder



Sample Images from IMAGEN_dataset Folder



Sample Images from GLIDE_dataset Folder

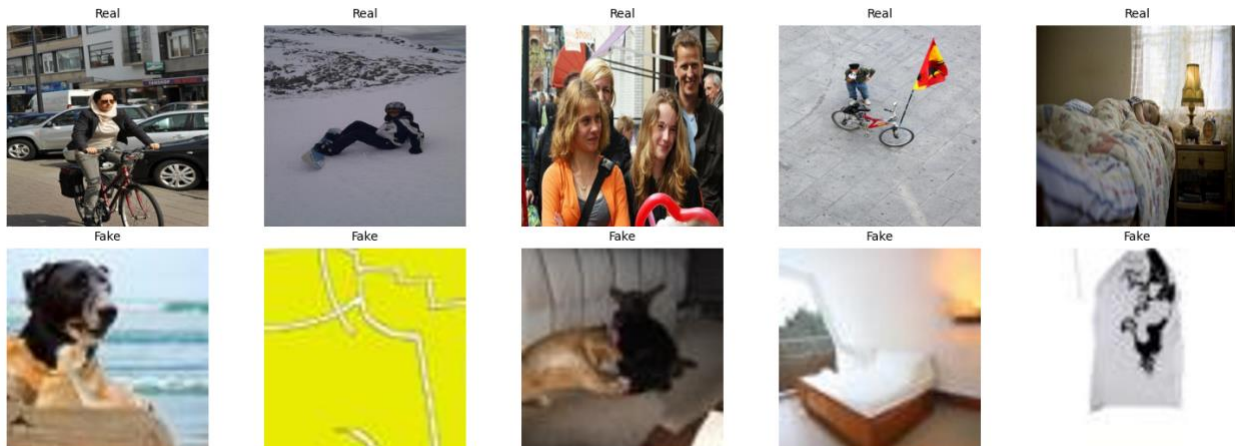


Figure 4-5 - Random images displayed from each of the subcategories (fake & real) of each image-generator folder.

- **Stable Diffusion:** Images are high quality, and differences between real and fake samples are subtle, often only apparent through inconsistencies in symmetry or object blending.
- **DALL·E:** The fake images tend to display either overly stylized illustrations (e.g., cartoon-like graphics) or subtle abnormalities in lighting and proportion. Some textures and object boundaries reveal minor inconsistencies that may aid model detection.
- **IMAGEN:** The synthetic outputs are visually compelling. However, finer details like human facial expressions or hands occasionally appear slightly unnatural.
- **GLIDE:** The lower resolution and blurriness in all the fake images make them more distinguishable. Real images seem vibrant and sharp, whereas fakes sometimes suffer from unclear object contours or strange lighting.

These visual clues highlight the varying complexity of detecting AI-generated images across different models. They also demonstrate that while some generative tools still leave behind noticeable artifacts, others produce content that is nearly indistinguishable from authentic images — underscoring the importance of building a model capable of generalising well across multiple sources.

5. DATA PREPARATION (CRISP-DM Stage 3)

Following initial data understanding, preparation steps were taken to ensure data quality and suitability for CNN modelling. The 'OrganizedDeepGuardDB_v2' dataset used in this analysis was prepared through initial data selection and under sampling. (Vidhya, 2019) Specifically, the 'GLIDE' subset was completely excluded from the broader source data due to their low image quality and to mitigate potential model bias towards artifacts specific to that generator. The remaining selected images were then converted to a uniform JPG format, assigned standardized names, and organized into distinct 'fake' and 'real' directories for subsequent pipeline processing (Figure 5.1).

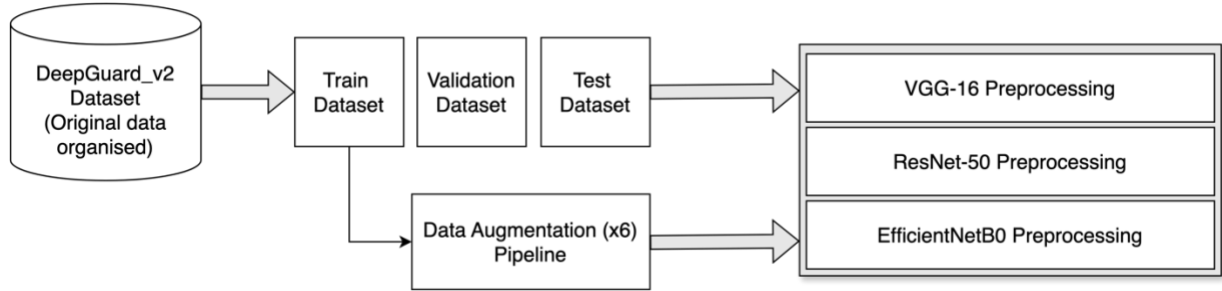


Figure 5-1 - Data preprocessing pipeline.

5.1. Construction of Training, Validation and Testing Sets

After organizing, the data was loaded using TensorFlow utilities from the organized folders. From the 11,995 total files, a standard partition was created to ensure independent data for model development, hyperparameter tuning (using validation), and final unbiased performance evaluation on the test set. This resulted in the following distribution:

- Training Set: 9,596 images (before augmentation)
- Validation Set: 1,200 images
- Test Set: 1,199 images

All datasets were processed using a batch size of 32. (dkarayiannakis, 2024) This resulted in 300 batches for the training set, 37 batches for the validation set, and 38 batches for the test set. To optimize I/O performance, these initial datasets were subsequently cached in memory after loading and resizing images to (224, 224) (TensorFlow, 2024b)

5.2. Data Augmentation

To improve model generalization and reduce the likelihood of overfitting on the finite training data, a strategy of on-the-fly data augmentation was applied exclusively to the training dataset pipeline (TensorFlow, 2024a). During model training, each training image underwent a series of randomly applied transformations before being fed to the model. These transformations, detailed in *Table 5.1*, included operations such as flips, rotations, zooms, and adjustments to brightness and contrast. The effectiveness and nature of these variations are illustrated in *Figure 5.2*.

Layer Used	Parameter(s)	Description & Purpose
Random Flip	mode="horizontal"	Randomly flips images horizontally.
Random Rotation	Factor = 0.1	Randomly rotates images
Random Zoom	Height = 0.1	Randomly zooms images vertically and horizontally by up to 10%.
Random Translation	Height = 0.1	Randomly shifts images vertically and horizontally by up to 10%.
Random Brightness	Factor = 0.15	Randomly adjusts image brightness within a +/- 15% range.
Random Contrast	Factor = 0.15	Randomly adjusts image contrast within a range.

Table 5.1 - Data augmentation layers and parameters used to enhance variability in the training dataset.

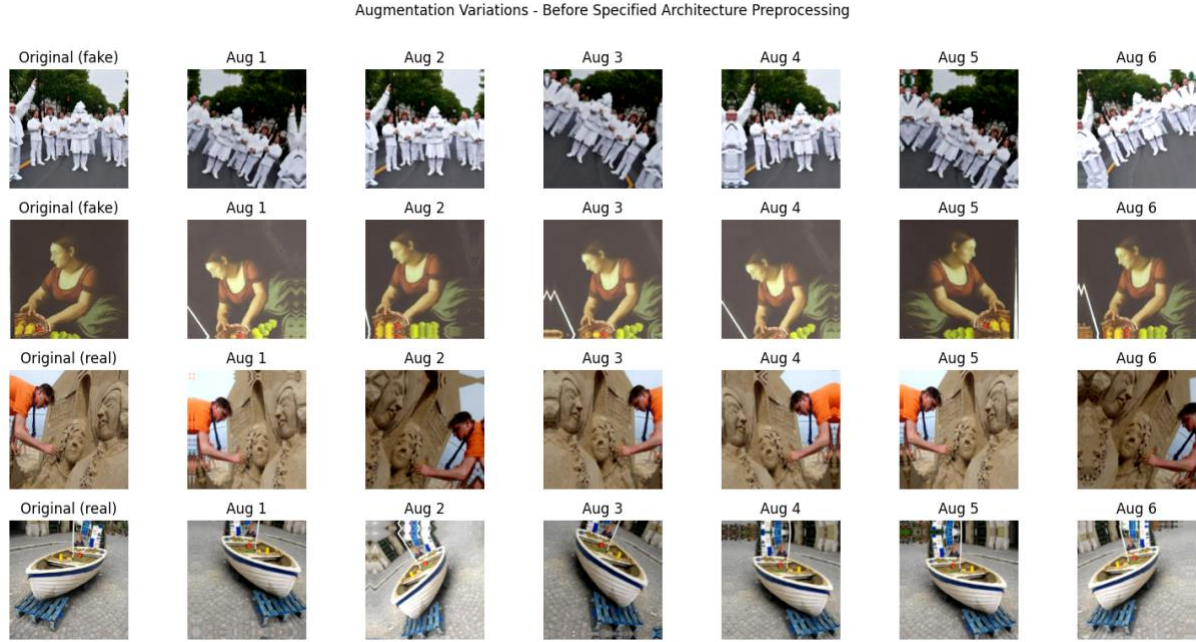


Figure 5-2 - Original images followed by six augmented variations.

5.3. Data Preprocessing for each Model

A critical requirement for leveraging pre-trained models via transfer learning is ensuring the input data conforms precisely to the format used during the model's original training. (Kromydas, 2023) As multiple distinct architectures (ResNet50, VGG16, EfficientNetB0) were evaluated, separate final processing stages were implemented for each. These stages applied the unique input transformations required by each specific architecture – encompassing necessary adjustments to pixel value scaling, colour channel ordering, or mean subtraction to their corresponding training, validation, and test datasets.

Finally, all data pipelines were optimised for efficient data delivery to the accelerator hardware by enabling asynchronous prefetching of upcoming batches. This comprehensive preparation process produced multiple, distinct datasets, each appropriately augmented, pre-processed for a specific target architecture, and optimised for efficient use in the comparative modelling phase.

6. MODELLING (CRISP-DM Stage 4)

According to the research (Cirillo, 2017), defining a data modelling strategy is commonly guided by the choice of the most popular alternatives. Probably a popular technique is popular because it is shown to be effective. We should focus on some kind of mix of these two components:

- How similar problems were solved in the past.
- Any emerging techniques which, given the type of data we have and the question we want to answer, could fit our purposes.

The go-to algorithms to use when working with image (unstructured) data fall under the category of Deep Learning. This is a subset of Machine Learning and an extension of a certain kind of algorithm called Artificial Neural Networks (ANNs). Leveraging the layers-architecture of ANNs, Deep Learning excels in learning different hierarchies of features across multiple layers. (Mirza Rahim Baig et al., 2020)

6.1. Convolutional Neural Networks

In our project, we decided to focus on the use of Convolutional Neural Networks. These are widely used for image classification, where the network will have to predict the right class (fake or real) for a given input. This type of Neural Networks uses convolution layers, which applies convolution operation in the input image followed by an activation function. To make such predictions, the last layer of CNN model needs to be a fully connected layer (dense) with the relevant activation function according to the type of prediction problem. As we are performing a binary classification, the activation function chosen is Sigmoid (its output is a numerical number ranging from 0 to 1 corresponding to the probability of the observation). (Mirza Rahim Baig et al., 2020)

6.1.1. CNN 101 Architecture

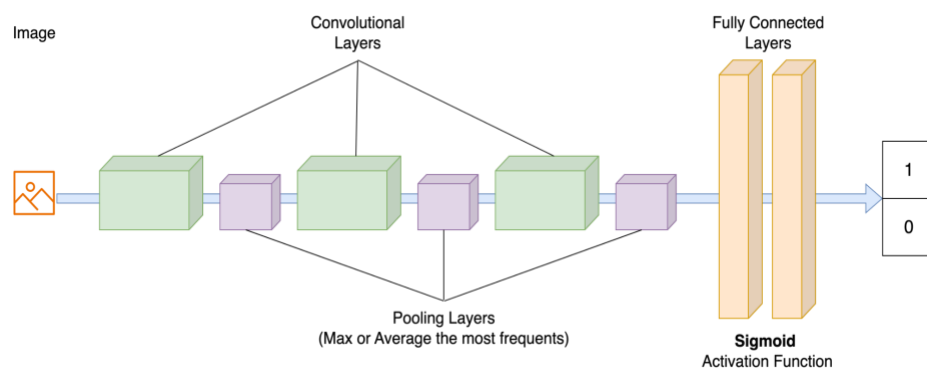


Figure 6-1 - Standard architecture of a standard CNN model.

Training a CNN model can be time-consuming. Determining the optimal number of filters (kernels), their sizes, and learning their weights through backpropagation is a complex and computationally intensive process. (Mirza Rahim Baig et al., 2020)

It would be great if we could reuse an existing architecture that has already been trained.

6.2. Transfer Learning

Luckily for us, such an option does exist, and it is called Transfer Learning. TensorFlow provides different implementations of state-of-the-art models that have been trained on the ImageNet dataset (over 14 million images). (Mirza Rahim Baig et al., 2020)

There are two types of transfer learning in the context of deep learning. In the feature extraction technique, a pretrained model on some standard dataset is used, but the top classification layer is removed. The new classifier layers are trained to perform classification. The pretrained model without the top classifier is treated as an arbitrary feature extractor to extract useful features from the new dataset. In the second approach, which is fine-tuning, the pretrained model weights are treated as the initial values for the new training, and these are updated in the training process. The goal when performing fine-tuning is to adapt the generic features to a given task rather than overwriting the generic learning. (Shazia et al., 2021).

Three CNN architectures were used in this project, chosen for their proven performance on large-scale image recognition tasks and for representing different architectural approaches—VGG-16 as a plain, deep network; ResNet-50 for its residual learning strategy (He et al., 2015); and EfficientNet for its compound scaling technique that optimises depth, width, and resolution (Tan and Le, 2019). By comparing their performance, we aim to evaluate how different transfer learning strategies affect our binary classification task.

6.3. VGG-16 Architecture

The VGG-16 model is a CNN architecture proposed by the Visual Geometry Group (VGG) at the University of Oxford. Its depth consists of 16 layers: 13 convolutional layers and 3 fully connected layers. It is renowned for its effectiveness and simplicity. VGG can achieve a strong performance across various computer vision tasks; in our case: image classification. (Simonyan and Zisserman, 2015)

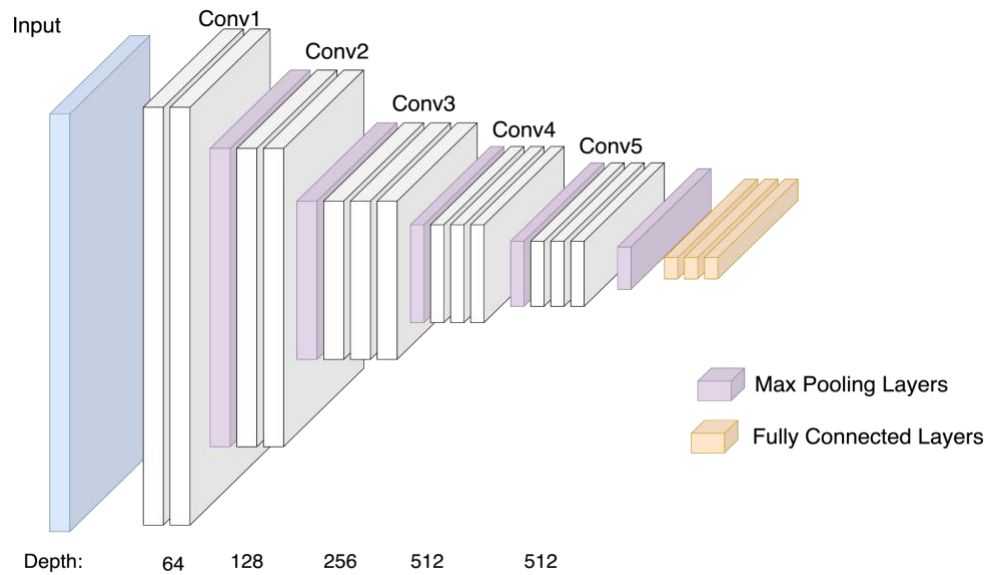


Figure 6-2 - Visual representation of VGG-16 architecture.

As illustrated in *Figure 6.2*, the architecture adopts a consistent design of stacked 3×3 convolutional layers followed by pooling operations, with progressively increasing filter depth. This hierarchical approach enables the extraction of increasingly abstract and discriminative features. Despite being more straightforward than newer architectures, VGG-16 remains a widely adopted baseline due to its robustness and ease of implementation. (Simonyan and Zisserman, 2015)

6.3.1. Degradation Problem

As networks become deeper, their accuracy initially improves but eventually begins to saturate and then degrade—not due to overfitting, but because deeper plain networks become harder to optimise. This limitation was identified by He et al. (2015), who observed that “*when deeper networks are able to start converging, a degradation problem has been exposed: with the network*

depth increasing, accuracy gets saturated and then degrades rapidly” (He et al., 2015, p.3). To address this, they introduced Residual Networks (ResNets), which use shortcut connections to allow gradients to flow more effectively during training.

6.4. ResNet-50 Architecture

ResNet-50 is a deep CNN architecture that introduced the concept of residual learning, allowing models to go significantly deeper without suffering from degradation in performance. Developed by He et al. (2015), ResNet-50 consists of 50 layers and is structured around residual blocks, which incorporate skip connections to facilitate efficient training of very deep networks (*Figure 6-9*). This architecture demonstrated state-of-the-art performance in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Compared to traditional plain networks, such as VGG-16, ResNet-50 provides better optimisation and generalisation performance while using fewer parameters at similar depth levels (He et al., 2015).

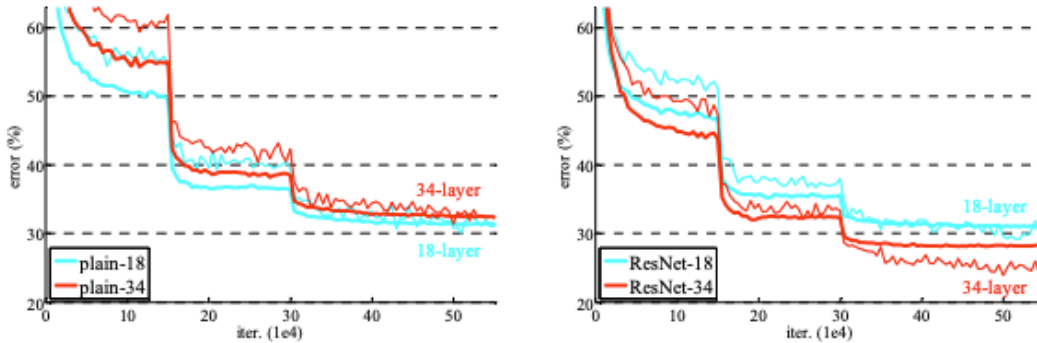


Figure 6-3 - Comparison of training and validation errors between plain networks and residual networks with 18 and 34 layers. Source: He et al. (2015)

The plot demonstrates that as depth increases, plain networks (left) suffer from higher training and validation error (degradation problem), while ResNet networks (right) maintain or improve performance, validating the effectiveness of residual learning.

6.4.1. Residual Block and Skip Connection

A residual block is a fundamental component of the ResNet architecture. Instead of learning a direct mapping $H(x)$, it learns a residual function $F(x) = H(x) - x$, which is then added back to the original input x . This result is the final output $F(x) + x$, as shown in *Figure 6.4*.

This design allows gradients to flow more easily through the network during backpropagation, addressing the vanishing gradient problem mentioned in the VGG-16 description and enabling the training of very deep networks without degradation in accuracy.

This mechanism was introduced by He et al. (2015), who demonstrated that learning residuals instead of direct mapping significantly improved training performance in deeper networks.

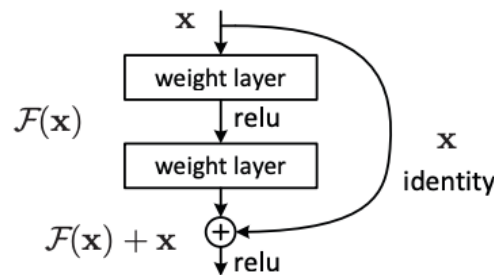


Figure 6-4 - Residual Learning Block. Improves gradient flow via a skip connection. Source: He et al. (2015)

6.5. EfficientNet Architecture

EfficientNet is a CNN architecture focused on the use of a compound scaling method to scale, in a uniform aspect, the depth, width and resolution of a ConvNet providing high accuracy with computational efficiency. So basically, a baseline ConvNet can be easily scaled to any target resource constraints, in a more principled and efficient way. (Tan and Le, 2020)

As the size and complexity of datasets grow, CNNs need to become deeper and more complex to achieve high accuracy. Scaling a model can be achieved in three ways: by increasing model depth, width, or image resolution.

The problem is that increasing the depth (by adding more layers) or the width (by adding more channels) causes an increase in the number of trainable parameters. As a result, it takes more time and computing power to train and make predictions.

Scaling also produces a Memory Bottleneck as larger models with more parameters require more memory to store the model weights and activations during processing.

6.5.1. Compound Scaling

According to Tan and Le (2020), different scaling dimensions (depth, width, image resolution) are not independent.

High-resolution images require deeper networks to capture large-scale features with more pixels. Additionally, wider networks are needed to capture the finer details present in these high-resolution images. To achieve better accuracy and efficiency, balancing all dimensions of network and resolution during ConvNet scaling is pivotal.

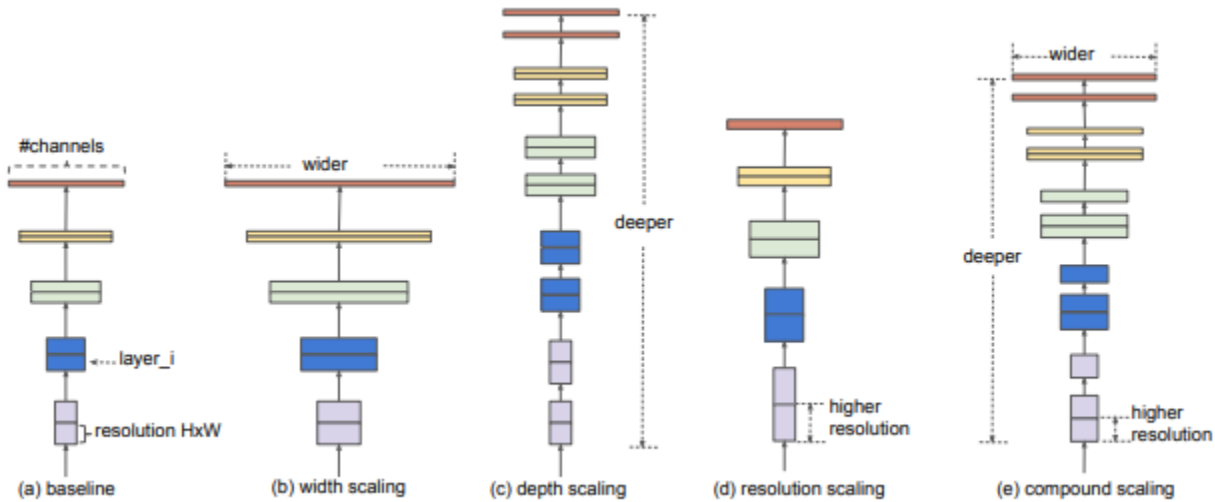


Figure 6-5 - Visual comparison of different ConvNet scaling strategies. Source: Tan, M. and Le, Q. (2020)

A compound scaling coefficient is used to uniformly scale all three dimensions (depth, width and resolution) in a proportional manner with the use of a predefined compound coefficient ϕ .

6.5.2. Main Components

As displayed in Figure 6.6, EfficientNet-B0 makes use of the MBConv block (Mobile Inverted Bottleneck Convolution) and the squeeze-and-excitation optimization.

The MBConv block is an evolved inverted residual block inspired by MobileNetv2.

An *inverted residual block* starts by expanding the input feature map into a higher-dimensional space then applies a depth wise convolution in this expanded space and finally projects the feature map back to a lower-dimensional space, the same as the input dimension. Why “inverted”?

Because this expansion of dimensionality at the beginning of the block and reduction at the end is opposite to the traditional approach where expansion happens towards the end of the residual block.

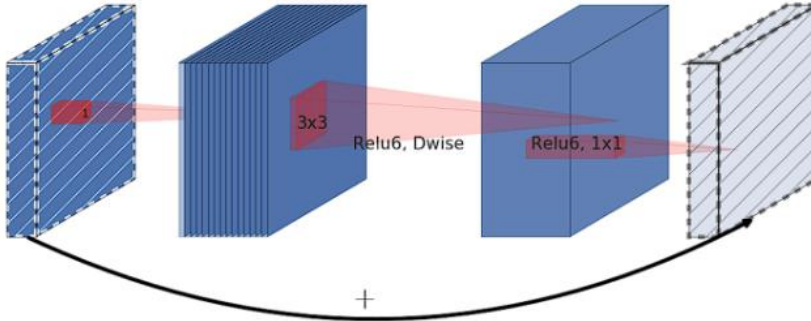


Figure 6-6 - Inverted Residual Block. Source: Sandler et al., (2019)

Squeeze-and-Excitation allows the model to emphasize useful features and suppress the less useful ones. It achieves this by performing two steps:

- Squeeze: the aggregation of the spatial dimensions of the feature maps across each channel into a single value using global average pooling, and
- Excitation: the model produces a collection of per channel weight using a fully-connected layer. The final step is to apply these learned scores to the original input feature map, channel-wise, effectively scaling each channel by its corresponding score.

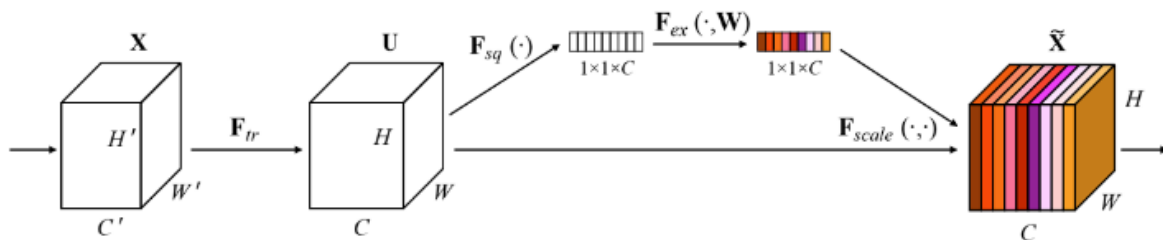


Figure 6-7 - Squeeze-and-Excitation Block. Source: Hu, (2019)

The convergence of these components –compound scaling, MBConv blocks, and squeeze-and-excitation– in the EfficientNet architecture results in a model that is both computationally efficient and highly accurate. This thoughtful combination enables EfficientNet to focus on the most relevant features within an image, as shown in the *Figure 6.8* class activation map. EfficientNet demonstrates how strategic architecture design can produce high performance without needless complexity by balancing model capacity and efficiency.

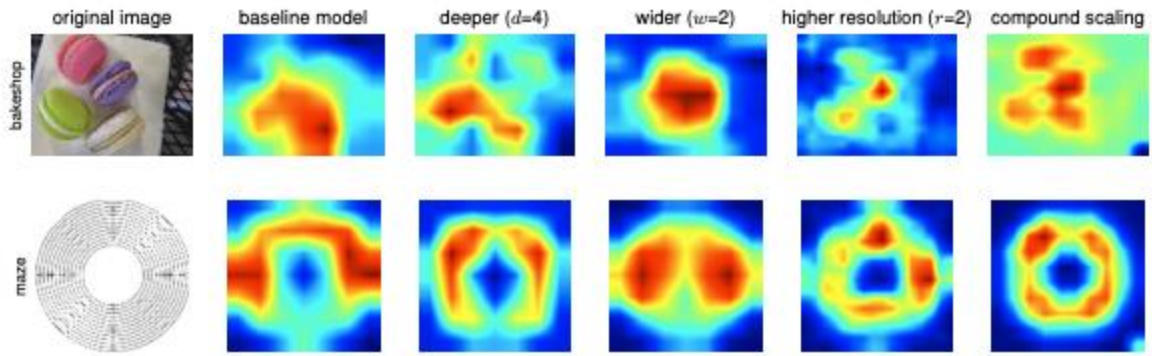


Figure 6-8 - Class Activation Map visualizing model's focus when making predictions. Source: Tan, M. and Le, Q. (2020)

6.6. Custom Classification Head

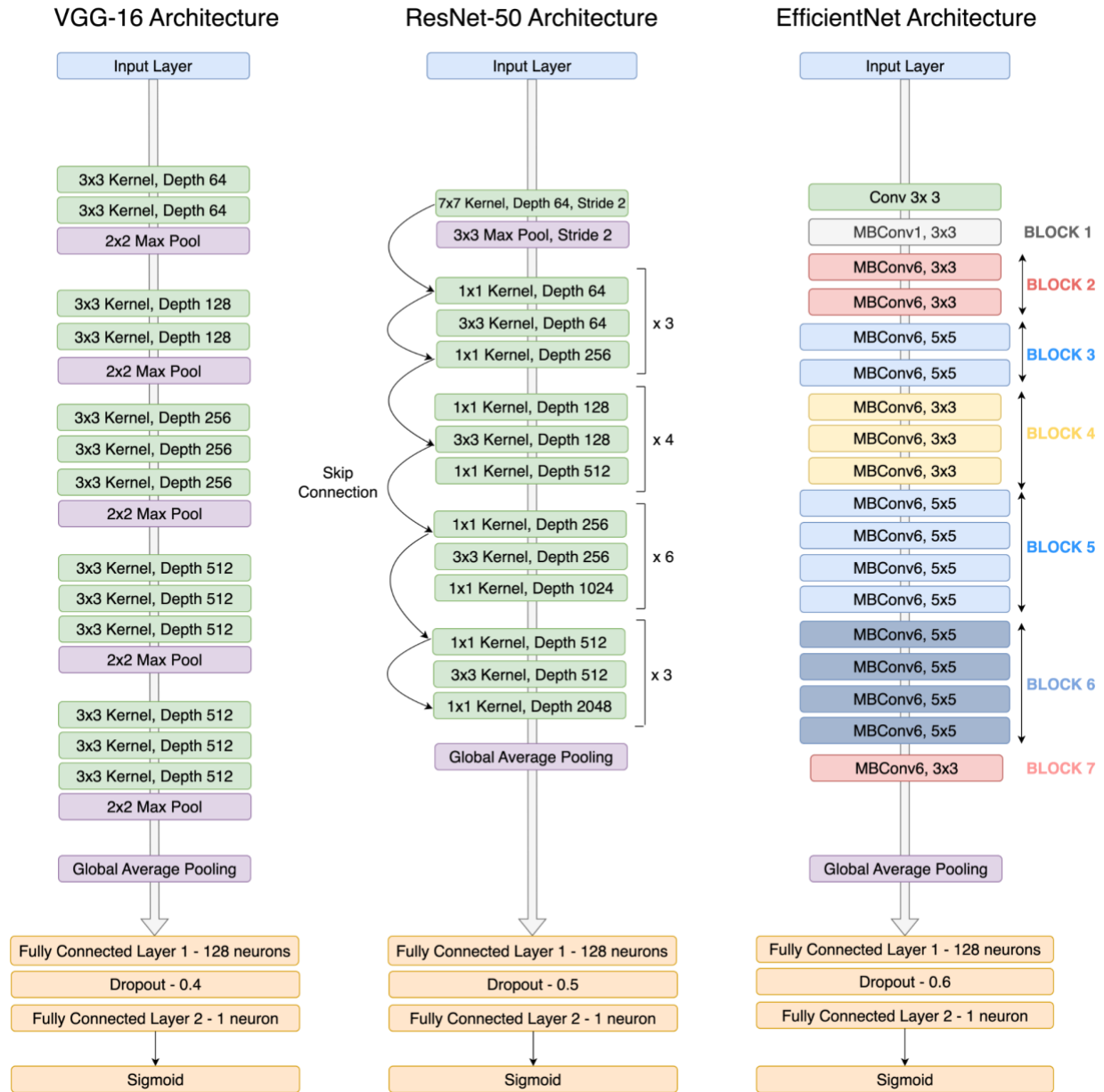


Figure 6-9 - Visual comparison of the three base model architectures followed by the established classification head for binary classification.

In all three transfer learning implementations– VGG-16, ResNet-50, and EfficientNet– we retained the pre-trained convolutional base as a fixed feature extractor and appended a custom classification head to adapt the models to our binary classification task. As illustrated in *Figure 6.9*, the original top classification layers were removed and replaced with two fully connected layers: the first using a ReLU activation to learn high-level feature combination and the second with Sigmoid activation

function to produce a probability for binary output. *Table 6.1* summarises the structure and function of these layers in a clear and simplified format.

Layer	Purpose	Number of Neurons	Activation Function
Global Average Pooling	Extract spatial information from feature maps.	-	-
Fully Connected Layer 1	Learn complex feature combinations.	128	ReLU
Batch Normalization	Apply transformation that maintains the mean output close to 0 and the output standard deviation close to 1.	-	-
Dropout	Regularisation technique that prevents overfitting by randomly deactivating neurons during training.	-	-
Fully Connected Layer 2	Produce final binary output.	1	Sigmoid

Table 6.1 - Displays details about the classification top layers of the neural network we configured using the different architectures.

6.7. Training Optimisation Strategies

To ensure efficient training and prevent overfitting or wasted computation, we employed two key callback mechanisms:

- **Callback:** object that can perform actions at various stages of training (e.g., start or end of an epoch)
- **EarlyStopping:** Stops training when a monitored metric has stopped improving. In our case, we chose to monitor `'val_loss'`, as the goal of the training was to minimise the loss. (Keras Team, n.d.)
- **ReduceLROnPlateau:** Reduce learning rate when a metric has stopped improving. If no improvement is seen for a 'patience' number of epochs, the learning rate is reduced. (Team, n.d.)

6.8. Conclusion

The modelling phase allowed us to explore and implement a variety of deep learning strategies tailored to our image classification task. We implemented three well-established deep learning architectures—VGG-16, ResNet-50, and EfficientNetB0—using transfer learning. Each architecture was adapted with a consistent classification head to suit our binary classification objective. These models now serve as the foundation for the next stage, where their performance will be evaluated.

7. EVALUATION (CRISP-DM Stage 5)

If we recall section 6.6 (Custom Classification Head), we have established the same classification layer across all base architectures while freezing their convolution layers to ensure a fair comparison of their feature extraction capabilities.

This approach allows us to assess which architecture produces the most informative feature maps for distinguishing "real" from "AI-generated" images.

This section summarises their comparative performance, drawing insights primarily from the confusion matrices, and a consolidated performance metrics comparison, training histories. The definitions of key metrics like accuracy, precision, recall, and F1-score, used in this evaluation, are summarised in *Table 7.1*.

CLASSIFICATION REPORT			
METRIC	FORMULA	DEFINITION	PURPOSE
Accuracy	$Accuracy = \frac{TP + TN}{Total}$	Proportion of correctly classified instances over the total number of instances.	How often the model gets things right, overall.
Precision	$Precision = \frac{TP}{TP + FP}$ $= \frac{TP}{Predicted\ Positive}$	The ratio of true positives to the sum of true and false positives	"For all instances classified positive, what percent was correct?"
Recall	$Recall = \frac{TP}{TP + FN}$ $= \frac{TP}{Actual\ Positives}$	The ability of a classifier to correctly find all positive instances	"For all instances that were actually positive, what percent was classified correctly?"
F1-Score	$F1\ Score = \frac{2 * Precision * Recall}{Precision + Recall}$	The F1 score is a weighted harmonic mean of precision and recall such that the best score is 1.0 and the worst is 0.0.	The weighted average of F1 should be used to compare classifier models, not global accuracy.

Table 7.1 - Summary of key classification metrics with formulas, definitions, and practical purposes. (Avinash Navlani, Fandango and Idris, 2021)

7.1. Confusion Matrices

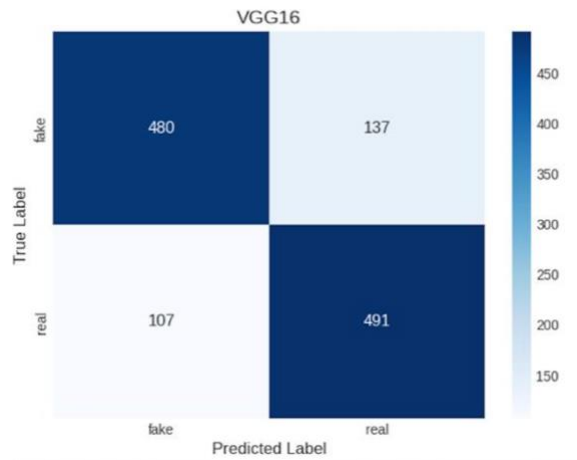


Figure 7-1 - VGG-16 Confusion Matrix

The VGG-16 model correctly classified 480 fake and 491 real images but showed a higher misclassification rate compared to other models, especially in identifying fake images (137 misclassified as real). This suggests that while the model has reasonable grasp of the task, it struggles with false negatives, which could pose risks in real-world detections scenarios.

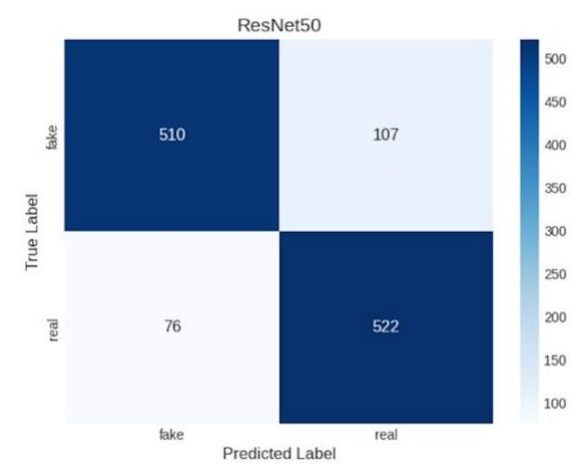


Figure 7-2 - ResNet-50 Confusion Matrix

ResNet-50 showed improved balance between both classes, correctly classifying 510 fake and 522 real images. With lower misclassification rates, especially for real images, it demonstrated stronger generalisation and higher reliability, making it more suitable candidate for real-world deployment compared to VGG-16.

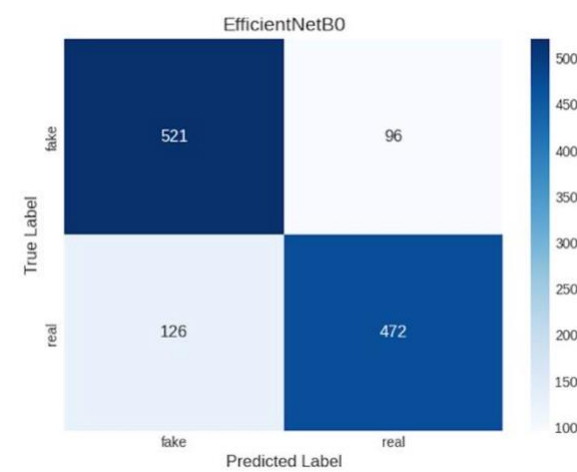


Figure 7-3 - EfficientNetB0 Confusion Matrix

EfficientNetB0 accurately predicted 521 fake and 472 real images. While it performed well in identifying synthetic content, it showed a higher number of false negatives for real images (126 misclassified). This indicates solid overall performance but slightly reduced reliability in detecting authentic content compared to ResNet-50.

7.2. Classification Report

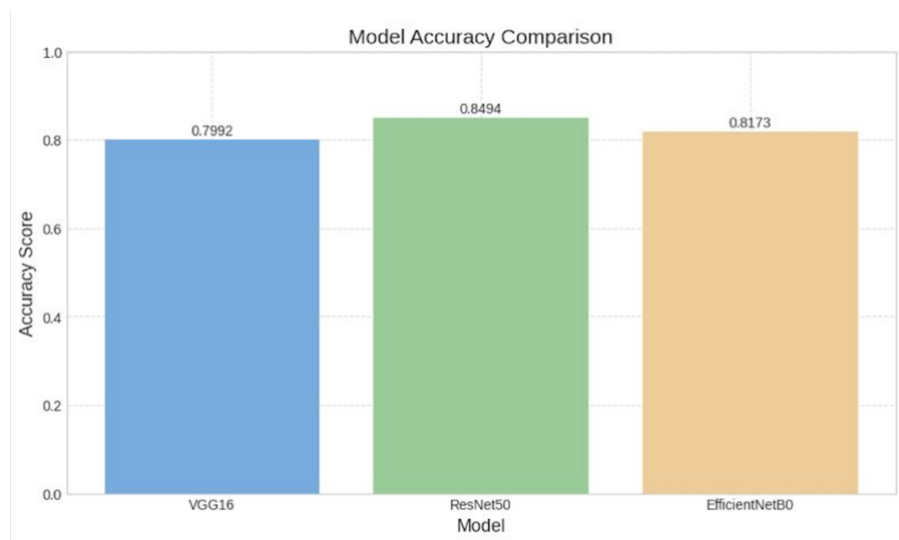


Figure 7-4 - Comparative accuracy scores of the three CNN architectures evaluated.

ResNet-50 achieved the highest accuracy (84.9%), followed by EfficientNetBO (81.7%) and VGG-16 (79.9%). These results highlight ResNet-50 as the most effective architecture for our binary classification task.

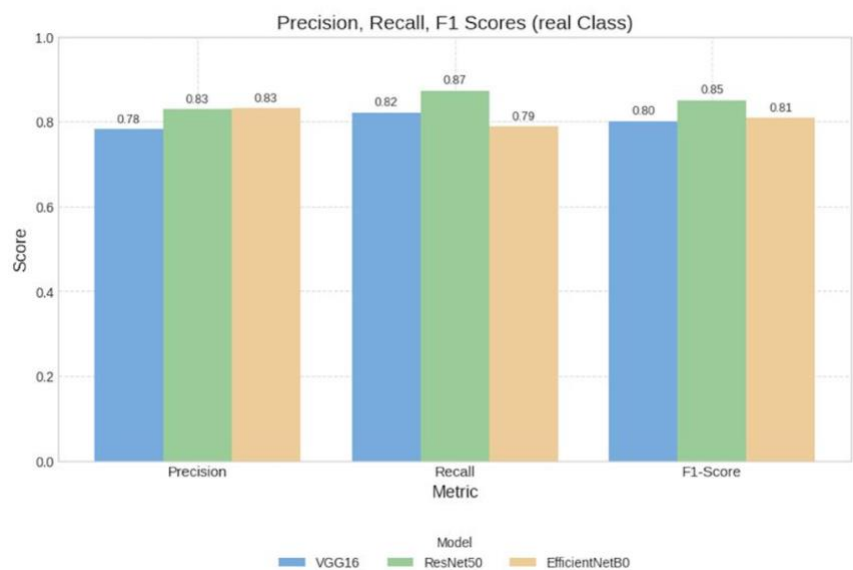


Figure 7-5 - Precision, Recall and F1-Score Comparison

ResNet-50 outperformed the other models across all metrics for the real class, achieving the highest recall (0.87) and F1-score (0.85). This indicates its strong ability to correctly identify real images, making it the most reliable model for detecting authentic content.

7.3. Training Dynamics

Comparative Training Histories of Models

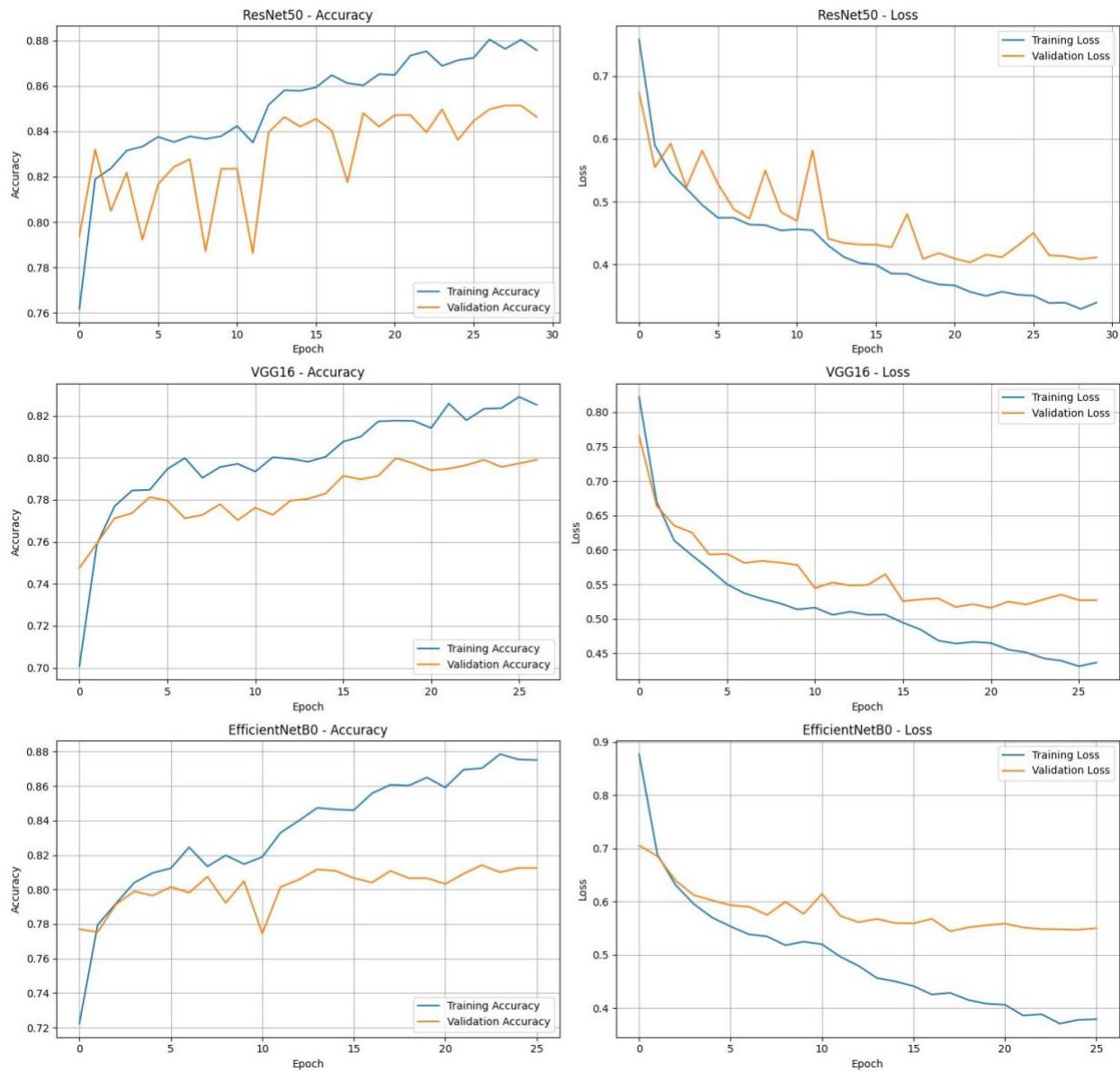


Figure 7-6 - Training and Validation accuracy and loss curves for all models over 30 epochs.

ResNet-50 displayed the most stable training behaviour, with closely aligned training and validation curves, indicating good generalisation and minimal overfitting. VGG-16 and EfficientNetB0 showed gaps between training and validation performance, suggesting moderate overfitting.

7.4. Model Selection

ResNet-50 clearly emerges as the most suitable architecture for this task. It not only delivered the highest F1-score (0.85) and recall (0.87) for the real class--indicating superior performance in correctly identifying authentic images--but also showed the most stable and consistent training dynamics, with minimal signs of overfitting. In contrast, VGG-16 and EfficientNetB0 demonstrated weaker generalisation and greater variability between training and validation performance. Based on both evaluation metrics and training behaviour, ResNet-50 was selected as the optimal model for further optimisation.

7.5. Model Optimisation: Hyperparameter Tuning and Fine-Tuning

Following the evaluation of the accuracy and generalisation of our transfer learning models (VGG-16, ResNet-50 and EfficientNetB0), ResNet-50 showed the best results. We used a two-step optimisation approach--Hyperparameter Optimisation (HPO) and fine-tuning-- to further enhance its predictive power.

This section aims to improve the performance of our best-performing model (ResNet-50) by refining its training process. We first use hyperparameter tuning to search the best configuration (e.g., learning rate, number of neurons, dropout rate), and then fine-tune the model by selectively unfreezing some of its top convolutional layers to adapt better to our dataset.

7.5.1. Hyperparameter Optimisation (HPO) - Keras Tuner

KerasTuner is an easy-to-use, scalable hyperparameter optimisation framework that solves the pain points of hyperparameter search. (Team, n.d.)

In this step, we utilize Keras Tuner's RandomSearch algorithm to identify the most effective combination of hyperparameters. The goal is to find a configuration that maximises validation accuracy while preventing overfitting.

The RandomSearch implementation was configured to train each model for up to 35 epochs, with early stopping set to a patience of 5, and max_trials value of 50, allowing for a broad exploration of hyperparameter combinations to maximise performance. The search process took approximately

10 hours and 27 minutes to complete. Additional implementation details are available in the submitted Jupyter Notebook and accompanying project repository.

Hyperparameter	Search Space	Best Value Found
Dense Layer	1, 2	1
Units per Dense Layer	64, 128, 256, 512	512 (layer 0)
Dropout Rate	0.3, 0.4, 0.5	0.5 (layer 0)
L2 Regularisation	0.0001, 0.001	0.001 (layer 0)
Optimizer	Adam', 'sgd'	Adam
Learning Rate	0.001, 0.0001, 0.0005	0.0001
Activation Function	ReLU	ReLU

Table 7.2 - Hyperparameter tuning results for ResNet-50 classification head using Keras Tuner. The search space included different configurations. The best-performing combination achieved a validation accuracy of 88.85%.

7.5.2. Fine-Tuning

Transfer Learning works because the features computed at the initial layers are more general and look similar. The features extracted in the top layers become more specific to the problem that we want to solve. (Zafar et al., 2018)

Fine-Tuning: unfreeze a few of the top layers of a frozen model base and jointly train both the newly added classifier layers and the last layers of the base model. This allows us to “fine-tune” the higher-order feature representations in the base model to make them more relevant for the specific task. (TensorFlow, 2025)

To further improve performance, one might want to repurpose the top-level layers of the pre-trained model to the new dataset via fine-tuning. In this case, we tuned the weights such that our model learned high-level features specific to the dataset. This technique is usually recommended when the training dataset is large and very similar to the original dataset that the pre-trained model was trained on. (TensorFlow, 2025)

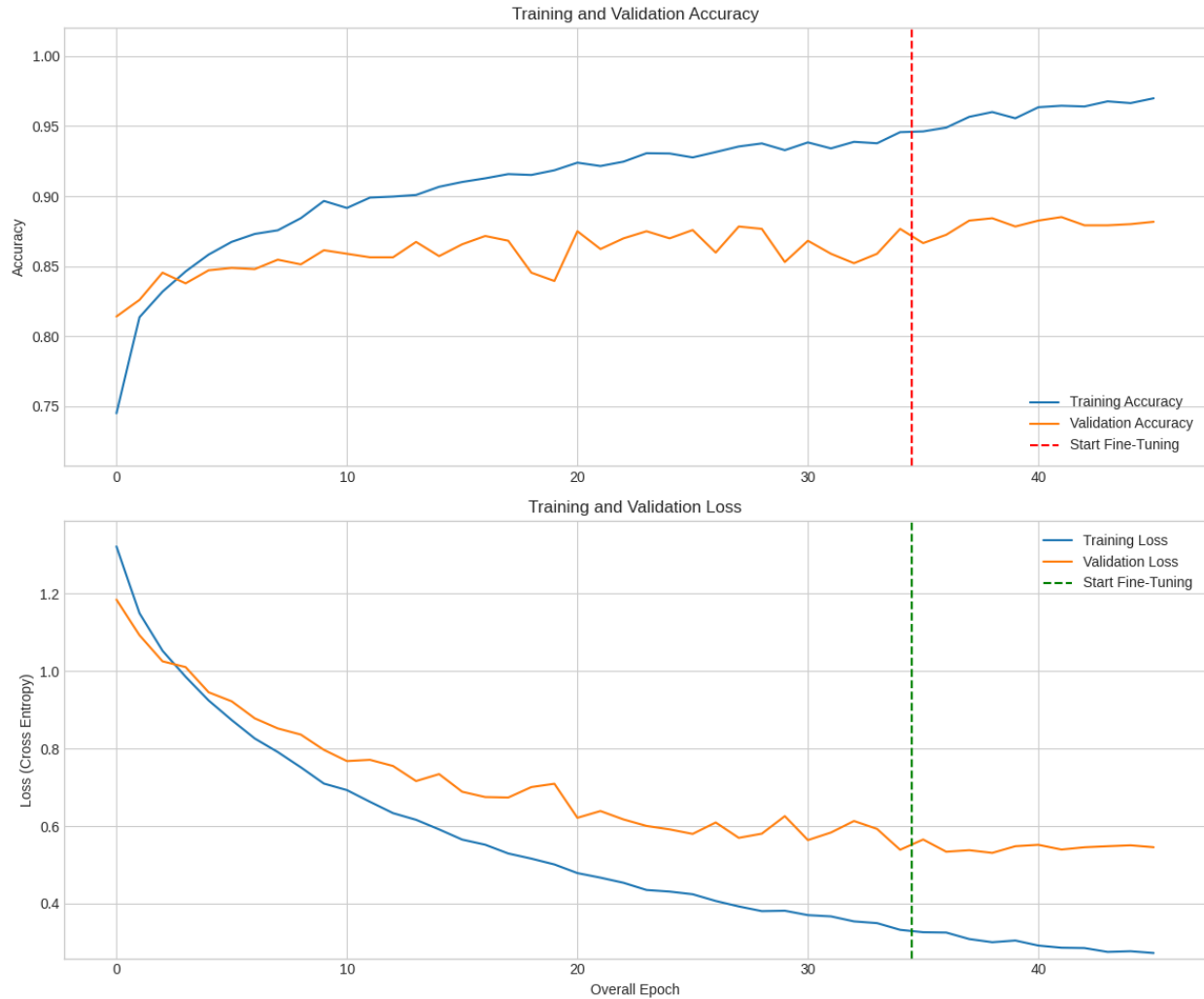


Figure 7-7 - Training and validation accuracy/loss curves for the fine-tuned ResNet-50 model.

The plots indicate a clear performance improvement after initiating fine-tuning. While the model showed solid learning during the head training phase, fine-tuning further reduced validation loss and increased validation accuracy. This confirms the effectiveness of unfreezing the top layers of the base model to adapt higher-level features to the specific classification task.

7.6. Final Evaluation

We started by comparing multiple CNN architectures using a shared classification head to evaluate feature extraction. This setup allowed a fair comparison of their feature extraction capabilities. ResNet-50 emerged as the best performer, demonstrating superior accuracy, generalisation, and training stability.

We then applied RandomSearch to optimise the classification layers, tuning key hyperparameters. Following this, fine-tuning was applied by unfreezing and retraining the top layers of the ResNet-50 base model. This process significantly improved model performance.

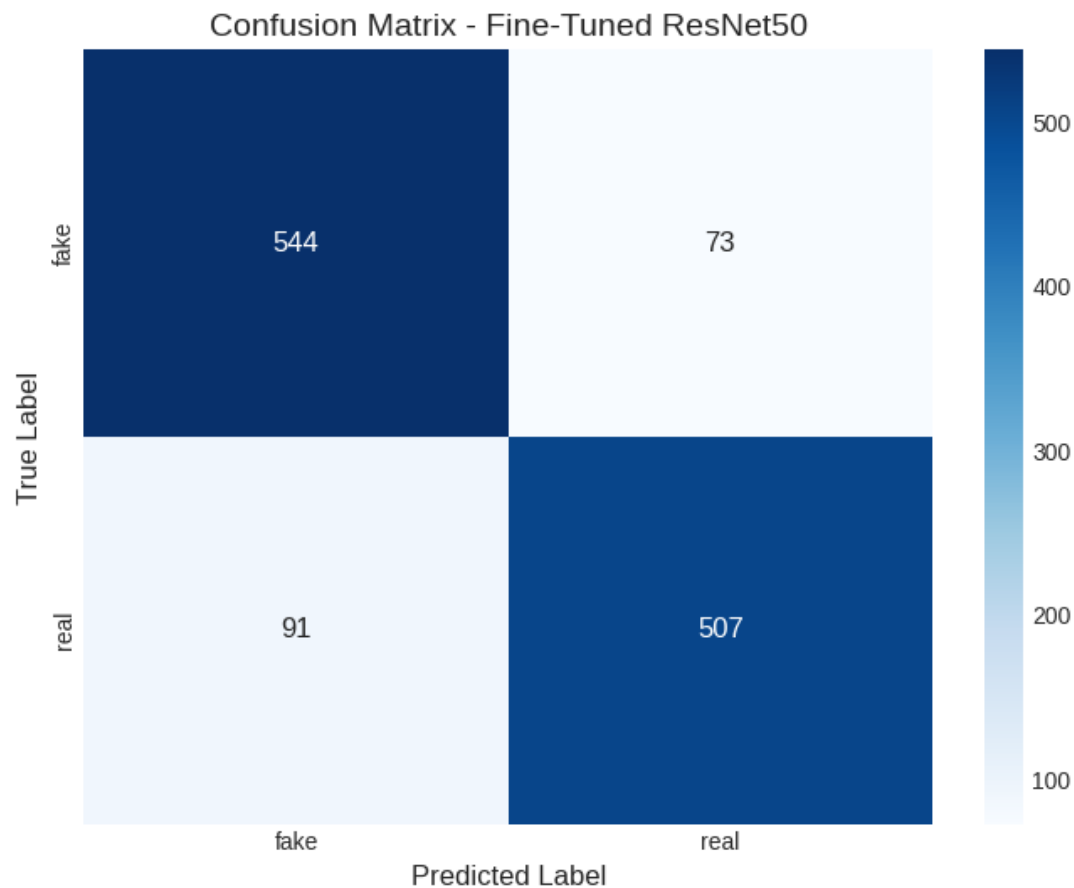


Figure 7-8 - Confusion matrix of the fine-tuned ResNet-50 model.

- **Accuracy: 86.5%** - representing the overall proportion of correct predictions.
- **Precision: 0.874** – how many predicted positives were actually correct.
- **Recall: 0.848** – how many actual positives were correctly identified.
- **F1-Score: 0.86** – the harmonic mean of precision and recall, indicating strong balance between precision and recall.

These results confirm that the fine-tuned ResNet-50 model offers robust and balanced performance, making it a strong candidate for real-world deployment scenarios.

8. Deployment (CRISP-DM Stage 6)

We have successfully developed a ResNet-50 based model capable of predicting whether an image is AI-generated or real with an overall accuracy of 86.5%. This model represents the final result of a structured process involving architecture comparison, hyperparameter optimisation, and fine-tuning of convolutional layers.

To make the model accessible and user-friendly, we deployed it through a Streamlit web application, chosen for its simplicity in building interactive, data-driven interfaces. The optimised model was exported from Google Colab in standard Keras format and integrated into the application, enabling users to upload images and receive classification results in real time.

The app was successfully tested with a range of unseen images, confirming its functionality and potential for real-world use.

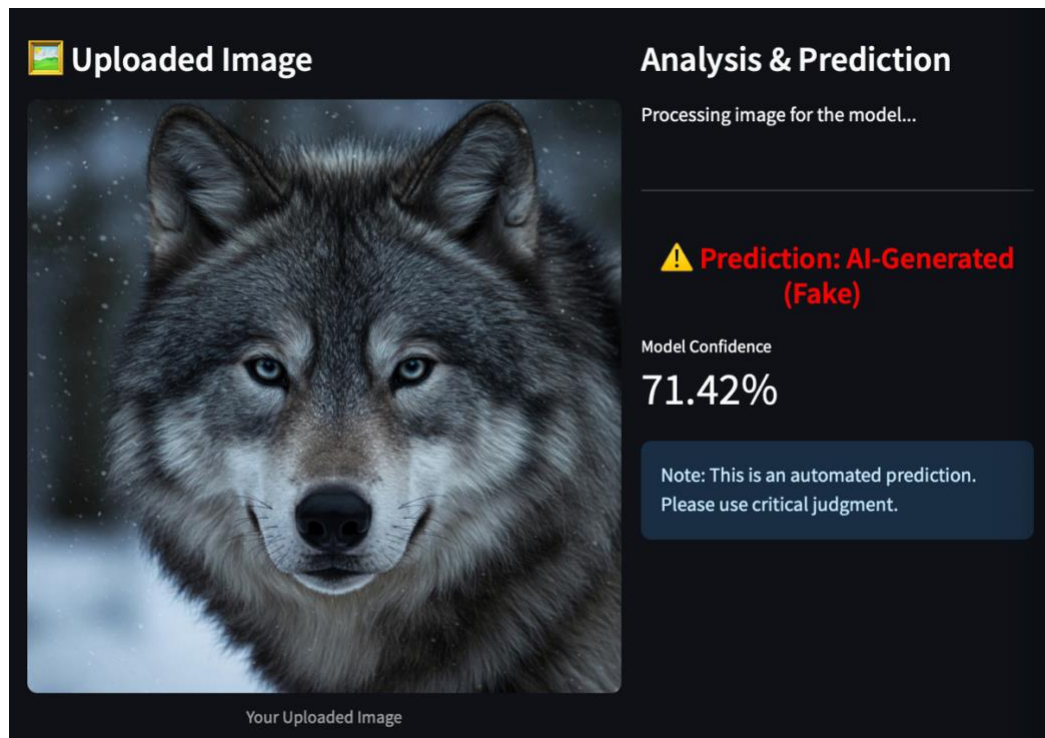


Figure 8-1- Streamlit Application – Prediction: AI-Generated

The screenshot illustrates the output of the deployed Streamlit web application using the fine-tuned ResNet-50 model. The model correctly identifies the uploaded image as AI-generated, with confidence of 71.42%.

This image was created using Imagen, a T2I generator. The result highlights the model's effectiveness in recognising subtle synthetic patterns even in highly realistic AI-generated content.

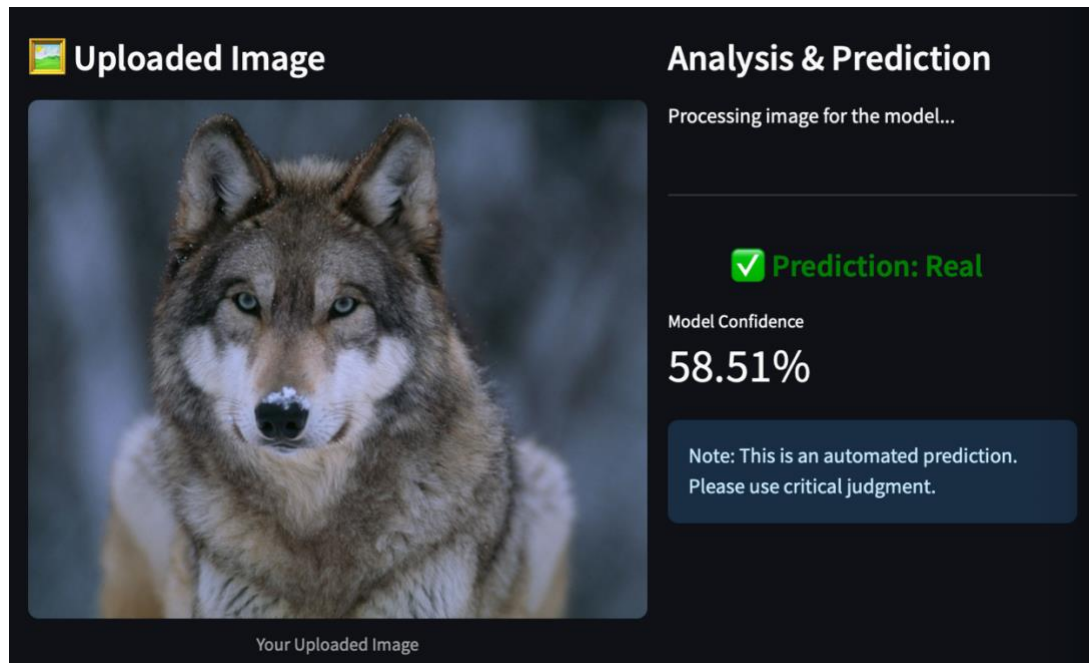


Figure 8-2 - Streamlit Application – Prediction: Real

The screenshot illustrates the output of the deployed web application using the fine-tuned ResNet-50 model. In this case, the model predicts the uploaded image as real with 58.51% confidence.

The image was sourced from National Geographic (Worrall, 2017). This example demonstrates the model's practical application in authentic image verification, even when distinguishing fine visual details.

9. Conclusion

This project successfully delivered a ResNet-50--based deep learning model designed to distinguish AI-generated images from real photographs—a growing challenge in the age of generative media. Following the CRISP-DM methodology, we structured the process across key stages: understanding the problem, preparing the dataset, evaluating multiple architectures, and deploying the best-performing model.

After comparing three CNN architectures—VGG-16, EfficientNetB0, and ResNet-50—ResNet-50 was selected for its superior balanced between accuracy, generalisation, and training behaviour. We further improved its performance through hyperparameter tuning and fine-tuning, resulting in a model that achieved 86.5% accuracy, along with high precision, recall and F1-score.

These results demonstrate that the model meets the defined [business success criteria](#):

- It consistently performs well across standard classification metrics.
- It keeps false positives and false negatives at reasonably low rate.
- It generalises effectively, as evidenced by its stable performance on unseen test data from diverse sources.
- Most importantly, the model was successfully deployed via a user-friendly Streamlit web application, showing clear potential for integration into real-world tools such as content verification systems, fake news detection, or moderation pipelines for social media platforms.

The research provides both a functional method to detect synthetic visual content and establishes a base for upcoming improvements. The future development of this work could involve increasing the dataset with additional AI generators, enhancing model generalisation capabilities and deploying the model through cloud-based APIs to support scalable platform use.

Appendix

Word Count

The word count was exceeded due to the inclusion of detailed comparisons between convolutional architectures. This comparative approach was proposed by our project supervisor to ensure a more comprehensive evaluation and to support a well-informed model selection process. Additionally, it was important for us to demonstrate a thorough understanding of how each architecture functions- we would not submit work we do not fully grasp. The extended discussion on model behaviour, evaluation metrics, and architectural differences was therefore necessary to uphold the academic and technical understanding expected for this project.

GitHub Repository Link

<https://github.com/CCT-College-Dublin/ca2-90-iav2002>

Original Dataset Link

https://drive.google.com/file/d/1-O7H-fG_kzTKHGad4L0FbtNJnNr7qGgo/view?usp=sharing

Reflective Journal

Bernardo Gandara

From the beginning, Ignacio and I agreed to take on a project that would challenge us intellectually and technically. We wanted to develop something that not only pushed our limits but also held real-world value.

Over the past four years at CCT College, I've worked full-time as a Community Operation Analyst for the world's biggest social media platform. This role exposed me daily to the challenges of content moderation and the rapidly increasing presence of AI-generated media. The opportunity to create a tool that could help me mitigate the risk of generative AI felt both meaningful and exciting. However, when we began this project, my knowledge of Convolutional Neural Networks (CNNs) was minimal.

What followed was a truly transformative experience. Each challenge and setback led me to research, learn, and overcome obstacles. I explored various CNN architectures, implemented model evaluation techniques, carried out hyperparameter optimisation and fine-tuning, and finally deployed the solution—leveraging the iterative nature of the CRISP-DM methodology every step of the way. This hands-on process turned my initial curiosity into real proficiency in CNN-based deep learning and deep enthusiasm for the subject.

I am now preparing to begin an internship as a Back-End Software Engineer, proud to close this academic chapter with a project that demanded the best of me. More importantly, it sparked a strong interest in pursuing a future career in Machine Learning.

I would like to express my deepest gratitude to all the lecturers who supported us throughout this journey—whose guidance was pivotal in completing this project successfully.

Finally, a special word of thanks to my “compañero de trinchera”, Ignacio. For the past four years, we’ve worked side by side—supporting, communicating, and learning from each other through every stage of our academic journey. This project was no exception. His dedication, collaboration, and friendship made this experience not only successful but truly memorable.

Ignacio Alarcon Varela

Since we started brainstorming about what type of project we wanted to do, we knew for a fact that image classification was our main idea, but at the same time we looked for a real problem to solve and we founded, which will only become bigger “AI-generated content”. Then personally speaking was a massive challenge since deep learning was a completely new field.

My primary focus on the practical implementation stages, from meticulous data preparation and setting the pipelines to have an easy integration of different models. To then start working on the implementation of them. All the work we did behind the scenes, from creating our own CNN, to then iterate countless times on the parameters of each Model. To then fight scenarios of overfitting for each model was an experience.

The hyperparameter tuning phase using KerasTuner for our ResNet50 model was particularly demanding. Monitoring those long training sessions. I vividly remember one search stretching over ten hours, it required a lot of patience, but it also offered a profound insight into the iterative nature of model optimization.

None of this would have been possible, or nearly as meaningful, without my project partner, Bernardo Gandara. He is a true friend and an outstanding collaborator whose dedication never wavered. Bernardo's thorough research and his strategic approach to optimization were foundational to our progress. His positive attitude and refusal to give up, especially when we hit roadblocks, consistently inspired me to keep exploring and problem-solving. Our constant communication and teamwork were the bedrock of this project.

Ultimately, this capstone has been more than just an academic requirement. Wrestling with the long hours of training and debugging, while tough, was worth every moment, the satisfaction of building a working AI application have been transformative, not just for the success of our project, but because it has genuinely sparked a deeper passion within me. This experience has solidified my desire to pursue a master's in data science and continue exploring this fascinating field. I'm proud of what Bernardo and I built and immensely grateful for the skills and inspiration gained along the way.

References

- Antoniadis, P. (2022). *Baeldung*. [online] Baeldung on Computer Science. Available at: <https://www.baeldung.com/cs/large-images-cnns>.
- Avinash Navlani, Fandango, A. and Idris, I. (2021). Python data analysis : perform data collection, data processing, wrangling visualization, and model building using Python. Birmingham: Packt Publishing.
- Babcock, J. and Bali, R. (2021). Generative AI with Python and TensorFlow 2 : harness the power of generative models to create images, text, and music. Birmingham: Packt Publishing, Limited.
- Brownlee, J. (2019). *Transfer learning in keras with computer vision models*. [online] Machine Learning Mastery. Available at: <https://machinelearningmastery.com/how-to-use-transfer-learning-when-developing-convolutional-neural-network-models/>.
- Cirillo, A. (2017). R data mining : implement data mining techniques through practical use cases and real-world datasets. Birmingham, UK: Packt Publishing.
- cocodataset.org. (n.d.). *COCO - Common Objects in Context*. [online] Available at: <https://cocodataset.org/#termsofuse>.
- Creative Commons (2018). *Creative Commons — Attribution 4.0 International — CC BY 4.0*. [online] Creativecommons.org. Available at: <https://creativecommons.org/licenses/by/4.0/legalcode>.
- dkarayiannakis (2024). *What is a common optimal batch size for training a convolutional neural network (CNN)? - EITCA academy*. [online] EITCA Academy. Available at: <https://eitca.org/artificial-intelligence/eitc-ai-dlpp-deep-learning-with-python-and-pytorch/convolution-neural-network-cnn/training-convnet/what-is-a-common-optimal-batch-size-for-training-a-convolutional-neural-network-cnn/>.
- He, K., Zhang, X., Ren, S. and Sun, J. (2015). Deep Residual Learning for Image Recognition. *arXiv*. doi:<https://doi.org/10.48550/arxiv.1512.03385>.
- Hu, J. (2019). *Squeeze-and-Excitation Networks*. [online] Available at: <https://arxiv.org/pdf/1709.01507>.
- Ikram Reghioa, Mouna Yasmine Namani, Gueltoum Bendiab, Mohamed Aymen Labiod, Stavros Shiaeles, 2024. DeepGuardDB: Real and Text-to-Image Synthetic Images Dataset. Available at: <https://dx.doi.org/10.21227/10ap-pk52>.
- Keras Team (n.d.). *Keras documentation: EarlyStopping*. [online] keras.io. Available at: https://keras.io/api/callbacks/early_stopping/ [Accessed 8 May 2025].

Kromydas, B. (2023). *Image classification using pre-trained ImageNet models in TensorFlow & keras*. [online] LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with code, & tutorials. Available at: <https://learnopencv.com/image-classification-pretrained-imagenet-models-tensorflow-keras/>.

Mirza Rahim Baig, Joseph, T.V., Nipun Sadvilkar, Mohan Kumar Silaparasetty and So, A. (2020). *The Deep Learning Workshop*. Packt Publishing Ltd.

Saleh, H. (2020). *The Deep Learning with PyTorch Workshop*. Packt Publishing Ltd.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2019). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. [online] Available at: <https://arxiv.org/pdf/1801.04381v4>.

Shazia, A., Xuan, T.Z., Chuah, J.H., Usman, J., Qian, P. and Lai, K.W. (2021). A comparative study of multiple neural network for detection of COVID-19 on chest X-ray. *EURASIP Journal on Advances in Signal Processing*, 2021(1). doi:<https://doi.org/10.1186/s13634-021-00755-1>.

Simonyan, K. and Zisserman, A. (2015). *VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION*. [online] Available at: <https://arxiv.org/pdf/1409.1556>.

Tan, M. and Le, Q. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. [online] Available at: <https://arxiv.org/pdf/1905.11946>.

Team, K. (n.d.). *Keras documentation: KerasTuner*. [online] keras.io. Available at: https://keras.io/keras_tuner/.

Team, K. (n.d.). *Keras documentation: ReduceLROnPlateau*. [online] keras.io. Available at: https://keras.io/api/callbacks/reduce_lr_on_plateau/.

TensorFlow (2019). *Load Images with tf.data | TensorFlow Core*. [online] TensorFlow. Available at: https://www.tensorflow.org/tutorials/load_data/images.

TensorFlow (2024a). *Data augmentation*. [online] TensorFlow. Available at: https://www.tensorflow.org/tutorials/images/data_augmentation#two_options_to_use_the_keras_preprocessing_layers.

TensorFlow (2024b). *Load and preprocess images*. [online] TensorFlow. Available at: https://www.tensorflow.org/tutorials/load_data/images#configure_the_dataset_for_performance.

TensorFlow. (2025). *Transfer learning with a pretrained ConvNet | TensorFlow Core*. [online] Available at: https://www.tensorflow.org/tutorials/images/transfer_learning.

Vidhya (2019). *Practical guide to deal with imbalanced classification problems in r*. [online] Analytics Vidhya. Available at: <https://www.analyticsvidhya.com/blog/2016/03/practical-guide-deal-imbalanced-classification-problems/>.

Worrall, S. (2017). *Why We're So Divided Over Saving Wolves*. [online] Animals. Available at: <https://www.nationalgeographic.com/animals/article/wolf-nation-brenda-peterson-wolves>.

Zafar, I., Tzanidou, G., Burton, R., Patel, N. and Araujo, L. (2018). *Hands-On Convolutional Neural Networks with TensorFlow : Solve Computer Vision Problems with Modeling in TensorFlow and Python*. Birmingham: Packt Publishing Ltd.